Colby Okazaki

(510)-561-5685 | colbyokazaki@gmail.com | linkedin.com/in/colby-okazaki | github.com/colby-okazaki

Education

University of California, Santa Cruz — Santa Cruz, California

Bachelor of Science in Computer Science — *Expected Graduation, Jun 2028*

- **GPA:** 3.66/4.00, *Dean's Honors*
- **Courses:** Data Structures and Algorithms, Principles of Computer Systems Design, Computer Systems and Assembly Language, Programming Abstractions: Python, Mathematical Thinking for Computer Science, Machine Learning: Data Analysis

Experience

Blueprint — Santa Cruz, California

Software Developer — *Oct 2026 – Present*

- Building a full-stack web application with **Next.js**, **Node.js**, and **PostgreSQL**, collaborating through **Git**
- Collaborating with a cross-functional team of developers and designers, owning and shipping individual features for a local nonprofit client serving **100+** people in the community

Technical Skills

Languages: TypeScript, JavaScript, Python, Java, C++, C, PostgreSQL, HTML/CSS
Frameworks & Tools: React, Next.js, Node.js, Express, Socket.io, Konva.js, Jest, React Testing Library, Prisma, OpenCV, PyTorch, REST APIs, Git, VS Code, Cursor, Claude Code, Vercel
Operating Systems: macOS, Linux

Projects

Group Restaurant Matcher | *TypeScript, Node.js, Express, Socket.io, REST APIs* — Nov 2026 – Present

- Built the back end of a group restaurant-matching app in **TypeScript**, **Express**, and **Socket.io**, where friends join with a 4-digit code, swipe yes or no on nearby places, and get a match when everyone agrees; co-designed the **REST API** contract with the front-end developer
- Implemented live room-based sync pushing member joins and matches to every phone, with reconnect catch-up, idempotent swipes safe to retry, and automatic cleanup of expired groups; covered the match rule with 10 automated tests
- Load tested with 1,200 concurrent live connections across 200 groups on a laptop: 0 failures, about **4,900** requests per second, and a **14.7ms** p95 from the final swipe to the match reaching the host's phone

Collaborative Whiteboard | *React, Konva.js, Socket.io, Node.js, Express, Jest* — Oct 2026 – Present

- Developed a real-time collaborative whiteboard in **React** and **Konva.js** where multiple users draw on a shared canvas, with a **Socket.io** backend syncing strokes across clients in room-based boards
- Implemented live multi-user cursors with a custom throttling utility that caps updates at 50ms while guaranteeing the final position is delivered, cutting network traffic during mouse movement
- Sent a full board snapshot to users joining mid-session so every client starts in the same state, with **Jest** and **React Testing Library** tests covering server state, timing logic, and UI components

Sports Stats Query Engine | *TypeScript, Next.js, React, REST APIs, NDJSON Streaming* — Oct 2026

- Built a full-stack sports stats app in **Next.js**, **React**, and **TypeScript** that answers plain-English questions with a one-sentence answer and stats table across 7 leagues, plus live scores, league pages, and per-game Gamecast and box scores, using ESPN's public **APIs**
- Designed a rule-based **NLP parser** extracting league, player/team, stat, season, and question type from **56** stat definitions with synonyms; answers all **52** benchmark questions across every supported type, streaming as **NDJSON**
- Cut response time up to **88%** via server-side **caching** of ESPN data (slowest requests: **1.2s** → **151ms**), with cache lifetimes tuned from 10 seconds for live games to 5 minutes for stats, while correctly declining out-of-scope queries

Movie Ticket Booking | *TypeScript, Node.js, Express, Prisma, PostgreSQL, React, JWT* — Jul 2026

- Built a full-stack cinema booking app with **JWT** auth, role-based user/admin portals, a **React** client for showtimes, seat selection, and tickets, plus an admin occupancy/revenue dashboard
- Modeled movies, showtimes, and seat inventory in **Prisma/PostgreSQL**, implementing transactional seat booking with conflict handling to prevent double-bookings
- Automated a daily Vercel Cron pipeline that syncs the real US theatrical lineup from the **TMDB API**, generating rolling showtimes and simulated seat demand through an idempotent sync that never touches customer bookings